Constrained Horn Clauses and their Applications

Lecture 2

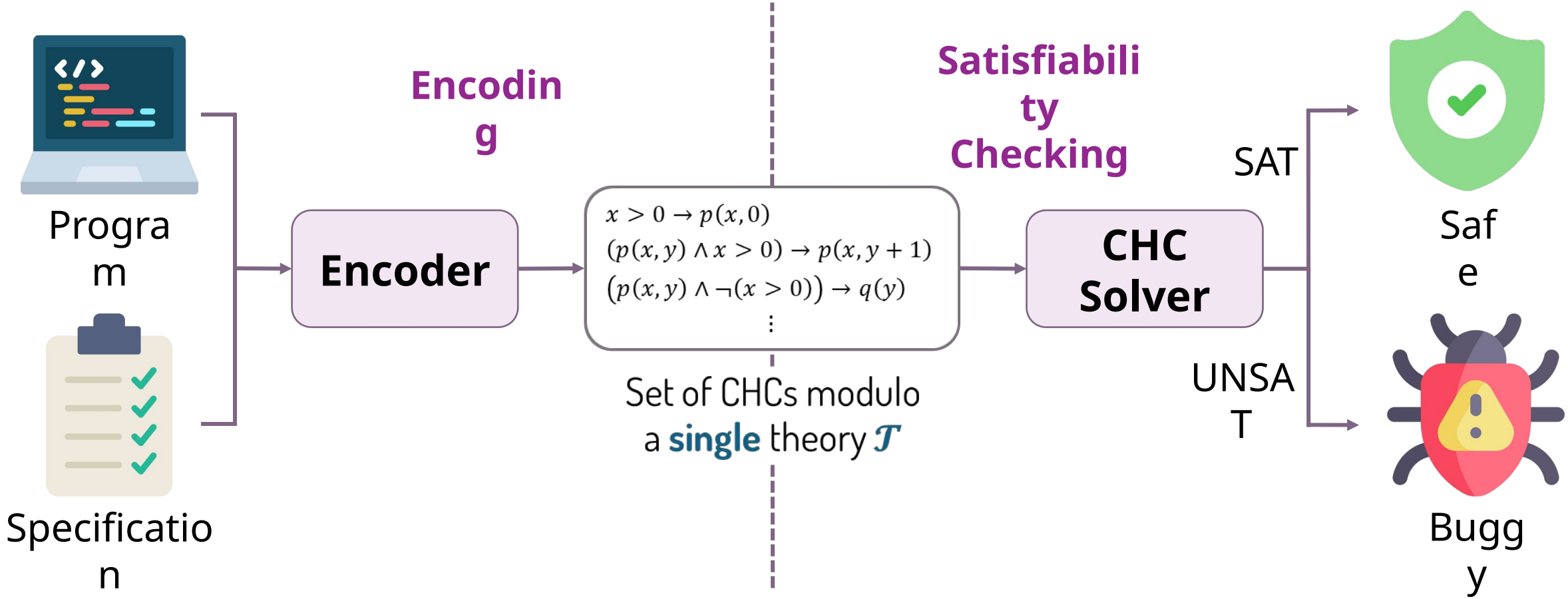
Orna Grumberg, Technion

Bit-Precise CHC Satisfiability Using Theory-Modular Reasoning

Omer Rappoport, Orna Grumberg, Yakir Vizel

ATVA 2026

Program verification via CHCs



Constrained Horn Clauses (CHCs)

A **constrained Horn clause** is a first order logic formula of the form:

$$\forall \bar{x}.\ \underbrace{\mathcal{B}[\bar{x}]}_{\text{body}} \rightarrow \underbrace{\mathcal{H}[\bar{x}]}_{\text{head}}$$

where
- $\mathcal{B}[\bar{x}] := p_1(\bar{x}_1) \wedge \cdots \wedge p_k(\bar{x}_k) \wedge \varphi$ (uninterpreted predicate; constraint)
- $\mathcal{H}[\bar{x}]$ is either $p_{k+1}(\bar{x}_{k+1})$ or φ'

Example: $\forall x, y, z.\ (p_1(x, y) \wedge p_2(y, z) \wedge z = x + y) \rightarrow p_3(x, z)$

Background theories

CHCs are formulated with respect to a background **theory**, which defines the sorts and semantics of the constraints.

Theory	Sorts	Constraint
Integer Arithmetic	Int	Theory / Sorts / Constraint Integer Arithmetic / Int / $z = x + y$ Real Arithmetic / Real / $x < y - 0.5$ Arrays / Array[I,V] / $A[i] = v$
Real Arithmetic	Real	Theory / Sorts / Constraint Integer Arithmetic / Int / $z = x + y$ Real Arithmetic / Real / $x < y - 0.5$ Arrays / Array[I,V] / $A[i] = v$
Arrays	Array[I,V]	Theory / Sorts / Constraint Integer Arithmetic / Int / $z = x + y$ Real Arithmetic / Real / $x < y - 0.5$ Arrays / Array[I,V] / $A[i] = v$

Bit-precise verification

In reality, machine integers are represented as finite binary arrays.

```
1 void f(int x) {
2   assume(x > 0);
3   int a = 0, b = 0;
4
5   while (a < x) {
6     a++; b--;
7   }
8
9   assert((a*b) < 0);
10 }
```

Example: 4-bit width

- **Domain**: $[-8,7]$
- **Input**: $x = 0011\ (3)$
- **After the loop**: $a = 0011\ (3), b = 1101\ (-3)$
- **Assertion**: fails

$$0011 \times 1101 = 0111\ (7)$$

overflow

Note: Integers behave differently than bit vectors

Bit-precise verification

In reality, machine integers are represented as finite binary arrays.

```
1 void f(int x) {
2   assume(x > 0);
3   int a = 0, b = 0;
4
5   while (a < x) {
6     a++; b--;
7   }
8
9   assert((a^b) < 0);
10 }
```

Example: 4-bit width

- **Domain**: $[-8,7]$
- **Input**: $x = 0011\ (3)$
- **After the loop**: $a = 0011\ (3), b = 1101\ (-3)$
- **Assertion**: holds

$$0011 \oplus 1101 = 1110\ (-2)$$

Option 1: encode modulo $\mathcal{T}_{\mathrm{B}}$

The theory of fixed-size bit-vectors

Not scalable

Spacer reaches width **9** in 2 hours

```
1 void f(int x) {
2   assume(x > 0);
3   int a = 0, b = 0;
4
5   while (a < x) {
6     a++; b--;
7   }
8
9   assert((a^b) < 0);
10 }
```

p → line 5 (while)

q → line 9 (assert)

$$\langle 0 \rangle \quad \left(x >_s^{\mathrm{B}} 0000 \wedge a = 0000 \wedge b = 0000\right) \rightarrow p(x, a, b)$$

$$\langle 1 \rangle \quad \left(p(x, a, b) \wedge a <_s^{\mathrm{B}} x\right) \rightarrow p\left(x, a +^{\mathrm{B}} 0001, b -^{\mathrm{B}} 0001\right)$$

$$\langle 2 \rangle \quad \left(p(x, a, b) \wedge \neg\left(a <_s^{\mathrm{B}} x\right)\right) \rightarrow q(a, b)$$

$$\langle 3 \rangle \quad q(a, b) \rightarrow \left(\left(a \oplus^{\mathrm{B}} b\right) <_s^{\mathrm{B}} 0000\right)$$

x, a, b have sort BV[4]

Option 2: encode modulo $\mathcal{T}_{\mathrm{I}}$

The theory of integer arithmetic

```
1  void f(int x) {
2    assume(x > 0);
3    int a = 0, b = 0;
4
5    while (a < x) {
6      a++; b--;
7    }
8
9    assert((a^b) < 0);
10 }
```

p → line 5 (while)

q → line 9 (assert)

$$\langle 0 \rangle \quad (x > 0 \wedge -8 \leq x < 8 \wedge a = 0 \wedge b = 0) \rightarrow p(x, a, b)$$

$$\langle 1 \rangle \quad (p(x, a, b) \wedge a < x) \rightarrow p\left(x, \mathrm{uts}_4\big((a+1) \bmod 16\big), \mathrm{uts}_4\big((b-1) \bmod 16\big)\right)$$

$$\langle 2 \rangle \quad \big(p(x, a, b) \wedge \neg(a < x)\big) \rightarrow q(a, b)$$

$$\langle 3 \rangle \quad q(a, b) \rightarrow (\mathrm{uts}_4(a \ \mathbf{XOR}\ b) < 0)$$

x, a, b have sort Int

$$\mathrm{uts}_4(t) = \mathrm{ite}(t < 8, t, t - 16)$$

Option 2: encode modulo $\mathcal{T}_{\mathrm{I}}$

The theory of integer arithmetic

Not scalable

Spacer reaches width **3** in 2 hours

```
1 void f(int x) {
2   assume(x > 0);
3   int a = 0, b = 0;
8
9   assert((a^b) < 0);
10 }
```

p →

q →

$$a \text{ XOR } b \coloneqq \sum_{i=0}^{n-1} 2^i \cdot \text{ite}\left(\left(\left(a \text{ div } 2^i\right) \bmod 2\right) \leftrightarrow \left(\left(b \text{ div } 2^i\right) \bmod 2\right), 0, 1\right)$$

$$\langle 0 \rangle \quad (x > 0 \wedge -8 \leq x < 8 \wedge a = 0 \wedge b = 0) \rightarrow p(x, a, b)$$

$$\langle 1 \rangle \quad (p(x, a, b) \wedge a < x) \rightarrow$$

$$\langle 3 \rangle \quad q(a, b) \rightarrow (\text{uts}_4(a \,\mathbf{XOR}\, b) < 0)$$

x, a, b have sort Int

$$\text{uts}_4(t) = \text{ite}(t < 8, t, t - 16)$$

Key idea: don't commit to one theory

Our approach: Leveraging the complementary strengths of both $\mathcal{T}_{\mathrm{B}}$ and $\mathcal{T}_{\mathrm{I}}$

```
1 void f(int x) {
2   assume(x > 0);
3   int a = 0, b = 0;
4
5   while (a < x) {
6     a++; b--;
7   }
8
9   assert((a^b) < 0);
10 }
```

p → line 5

q → line 9

$$\langle 0 \rangle \quad (x > 0 \wedge -8 \leq x < 8 \wedge a = 0 \wedge b = 0) \rightarrow p(x, a, b)$$

$$\langle 1 \rangle \quad (p(x, a, b) \wedge a < x) \rightarrow p\left(x, \mathrm{uts}_4\big((a+1) \bmod 16\big), \mathrm{uts}_4\big((b-1) \bmod 16\big)\right)$$

$$\langle 2 \rangle \quad \big(p(x, a, b) \wedge \neg(a < x)\big) \rightarrow q(a, b)$$

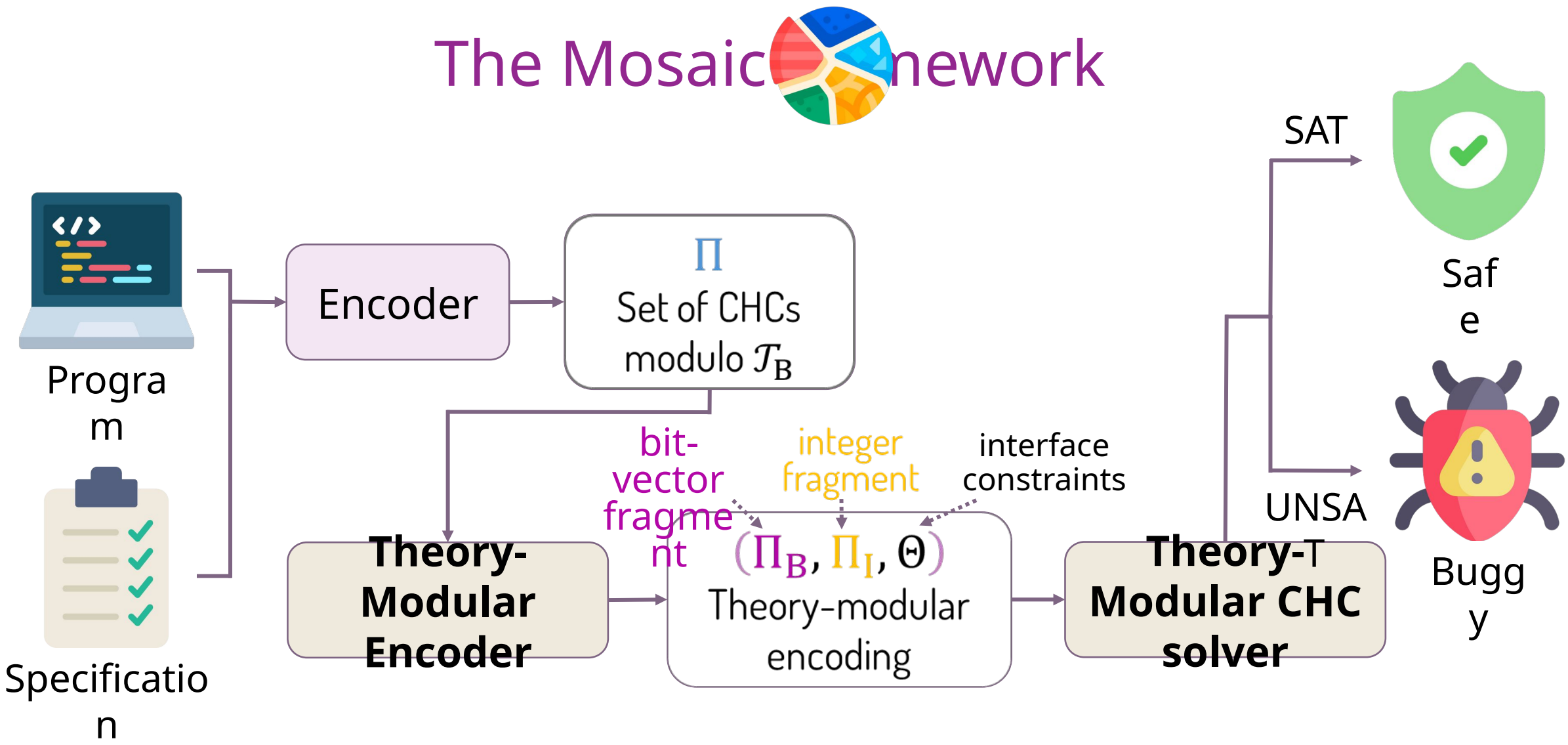
sort Int

interface constraint

$$\langle 3 \rangle \quad q(a, b) \rightarrow \left(\left(a \oplus^{\mathrm{B}} b\right) <_{\mathrm{s}}^{\mathrm{B}} 0000\right)$$

sort BV[4]

The Mosaic Framework



Relating values across theories

- Each bit-vector variable is associated with an integer counterpart.

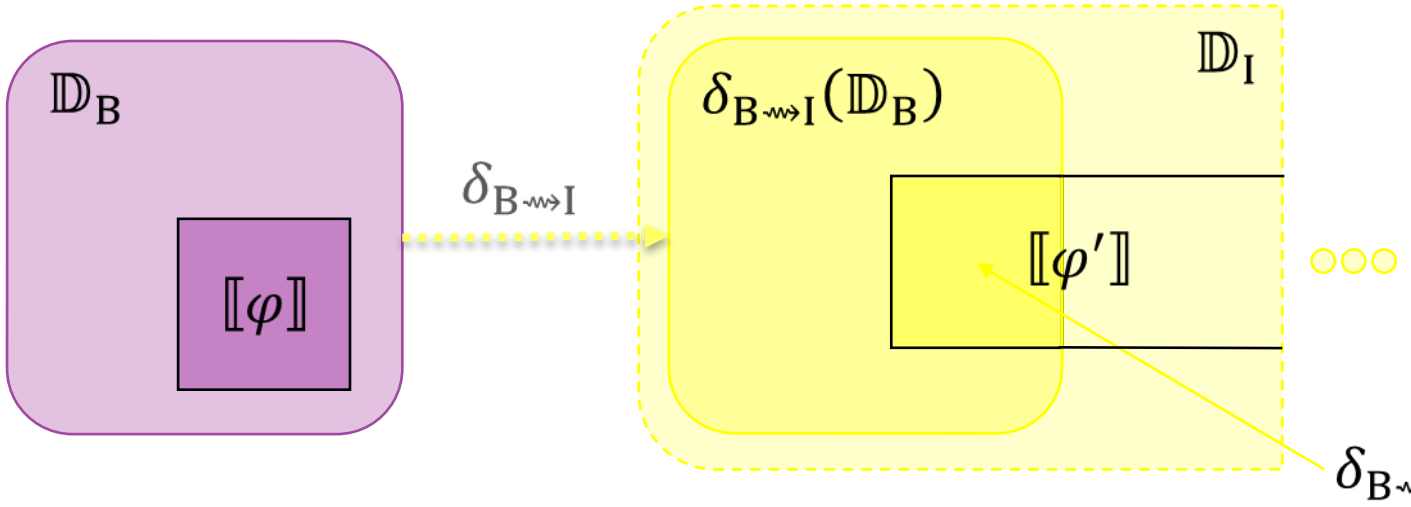
  Example: x: BV[4] ↔ x': Int

- For each variable pair, an injective **domain transformer** $\delta_{\mathrm{B} \rightsquigarrow \mathrm{I}}$ maps each bit-vector value to an integer. Two natural choices:
  - Unsigned: image $[0,15]$ $\delta_{\mathrm{B} \rightsquigarrow \mathrm{I}}(1101) = 13$
  - Signed: image $[-8,7]$ $\delta_{\mathrm{B} \rightsquigarrow \mathrm{I}}(1101) = -3$

What makes a translation correct

- A **theory transformer** $\tau_{\mathrm{B}\rightsquigarrow\mathrm{I}}$ from $\mathcal{T}_{\mathrm{B}}$ to $\mathcal{T}_{\mathrm{I}}$ translates every quantifier-free bit-vector formula φ into a quantifier-free integer formula $\varphi' = \tau_{\mathrm{B}\rightsquigarrow\mathrm{I}}(\varphi)$ such that:

$$\delta_{\mathrm{B}\rightsquigarrow\mathrm{I}}([\![\varphi]\!]) = ([\![\varphi']\!])\restriction_{\mathrm{B}}$$



$\varphi := x >_s^{\mathrm{B}} 0101$

$\varphi' := x' > 5$

"φ and φ' must agree on the restricted bit-vector domain"

- $\tau_{\mathrm{I}\rightsquigarrow\mathrm{B}}$ is defined dually.

$\tau_{\mathrm{B} \rightsquigarrow \mathrm{I}}$: bit-vectors to integers

- **Theory transformer**: Translates QF_BV formulas to QF_IA formulas.
- We adopt the int-blasting translation [Yoni Zohar et al., VMCAI'22].

  Example: $a +^{\mathrm{B}} b \mapsto (a' + b') \bmod 2^n$

- The one difference: we extend int-blasting to support signed domain transformers.
  - Adjusted domain restrictions
  - Signed ⟷ unsigned conversions

$\tau_{\mathrm{I} \rightsquigarrow \mathrm{B}}$: integers to bit-vectors

Based on **syntactic substitution**:

$$x' < y' + 1 \;\mapsto\; x <_s^{\mathrm{B}} y +^{\mathrm{B}} 0001$$

Problem: overflow changes the semantics

- Consider the integer assignment:

$$x' = 7, y' = 7$$

- The integer formula is satisfied:

$$7 < 7 + 1$$

- Corresponding bit-vector assignment:

$$x = 0111, y = 0111$$

- The bit-vector formula is not satisfied:

$$0111 \not<_s^{\mathrm{B}} 0111 +^{\mathrm{B}} 0001 \; (= -8)$$

$\tau_{\mathrm{I} \rightsquigarrow \mathrm{B}}$: integers to bit-vectors

Based on **syntactic substitution**:

$$x' < y' + 1 \qquad \mathrm{sext}(x, 1) <_s^{\mathrm{B}} \mathrm{sext}(y, 1) +^{\mathrm{B}} 00001$$

Problem: overflow changes the semantics

- Consider the integer assignment:

$$x' = 7, y' = 7$$

- The integer formula is satisfied:

$$7 < 7 + 1$$

- Corresponding bit-vector assignment:

$$x = 0111, y = 0111$$

- The bit-vector formula is not satisfied:

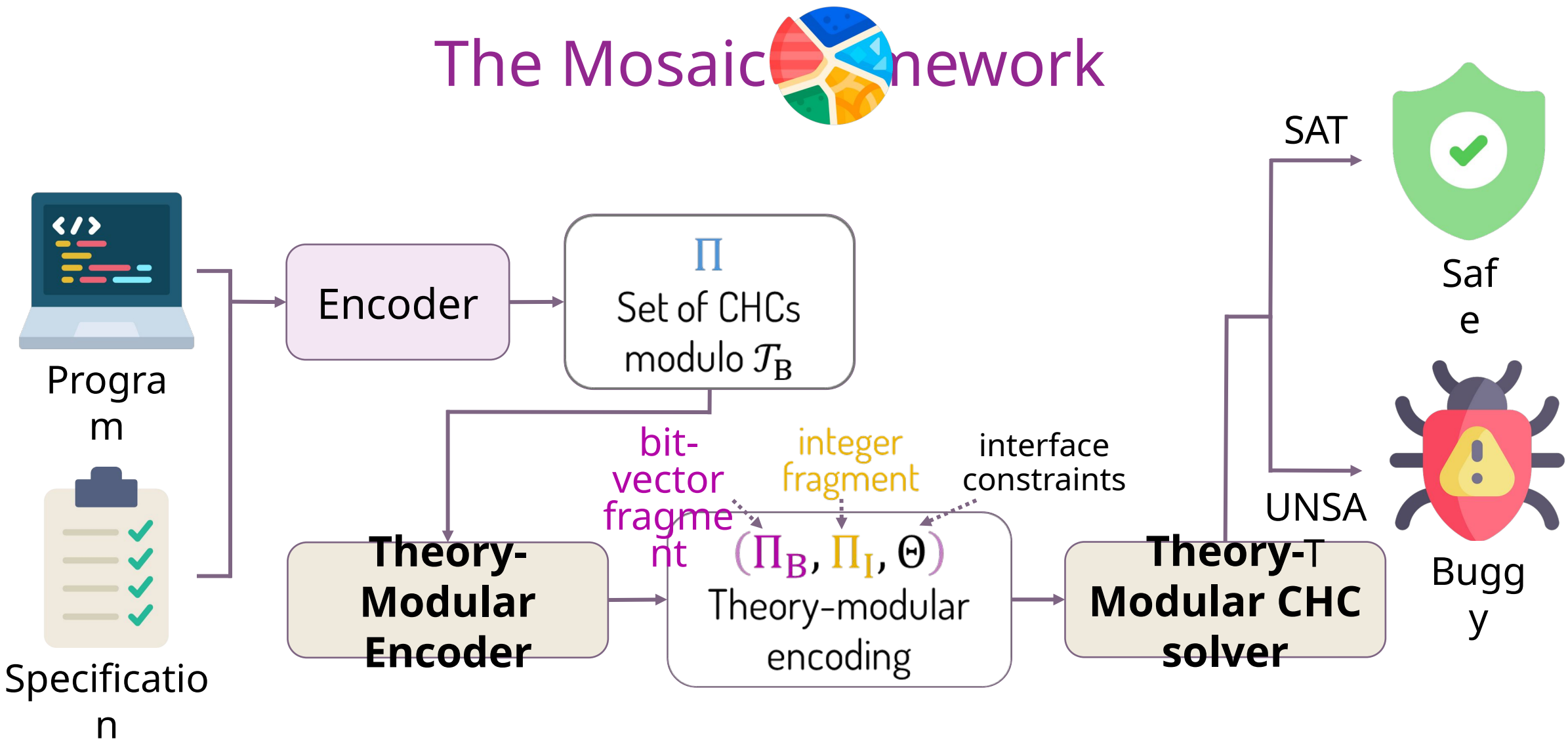
$$0111 \not<_s^{\mathrm{B}} 0111 +^{\mathrm{B}} 0001 \ (= -8)$$

Solution: sign-extend variables and constants by the minimal width that prevents overflow.

- The bit-vector formula is satisfied:

$$00111 <_s^{\mathrm{B}} 00111 +^{\mathrm{B}} 00001 \ (= 8)$$

The Mosaic Framework



Encoding, step 1: partition

Π:

$\langle 0 \rangle \quad \left(x >_{\mathrm{s}}^{\mathrm{B}} 0000 \wedge a = 0000 \wedge b = 0000\right) \rightarrow p(x, a, b)$

$\langle 1 \rangle \quad \left(p(x, a, b) \wedge a <_{\mathrm{s}}^{\mathrm{B}} x\right) \rightarrow p\left(x, a +^{\mathrm{B}} 0001, b -^{\mathrm{B}} 0001\right)$

$\langle 2 \rangle \quad \left(p(x, a, b) \wedge \neg\left(a <_{\mathrm{s}}^{\mathrm{B}} x\right)\right) \rightarrow q(a, b)$

$\langle 3 \rangle \quad q(a, b) \rightarrow \left(\left(a \oplus^{\mathrm{B}} b\right) <_{\mathrm{s}}^{\mathrm{B}} 0000\right)$

Encoding, step 1: partition

→ integer fragment

$$\langle 0 \rangle \quad \left(x >_s^{\mathrm{B}} 0000 \wedge a = 0000 \wedge b = 0000\right) \rightarrow p(x, a, b)$$

$$\langle 1 \rangle \quad \left(p(x, a, b) \wedge a <_s^{\mathrm{B}} x\right) \rightarrow p\left(x, a +^{\mathrm{B}} 0001, b -^{\mathrm{B}} 0001\right)$$

$$\langle 2 \rangle \quad \left(p(x, a, b) \wedge \neg\left(a <_s^{\mathrm{B}} x\right)\right) \rightarrow q(a, b)$$

→ bit-vector fragment

$$\langle 3 \rangle \quad q(a, b) \rightarrow \left(\left(a \oplus^{\mathrm{B}} b\right) <_s^{\mathrm{B}} 0000\right)$$

Heuristic: clauses whose function applications are purely arithmetic → integer fragment

Encoding, step 2: the bit-vector fragment

→ integer fragment

$$\langle 0 \rangle \quad \left(x >_s^{\mathrm{B}} 0000 \wedge a = 0000 \wedge b = 0000\right) \rightarrow p(x, a, b)$$

$$\langle 1 \rangle \quad \left(p(x, a, b) \wedge a <_s^{\mathrm{B}} x\right) \rightarrow p\left(x, a +^{\mathrm{B}} 0001, b -^{\mathrm{B}} 0001\right)$$

$$\langle 2 \rangle \quad \left(p(x, a, b) \wedge \neg\left(a <_s^{\mathrm{B}} x\right)\right) \rightarrow q(a, b)$$

Π_{B}:

$$\langle 3 \rangle \quad q^{\mathrm{B}}(a, b) \rightarrow \left(\left(a \oplus^{\mathrm{B}} b\right) <_s^{\mathrm{B}} 0000\right)$$

i. Predicate renaming

Encoding, step 3: the integer fragment

i. Predicate renaming
ii. Variable renaming
iii. Constraint translations
iv. Domain restrictions

Π_{I}:

$x' \upharpoonright_{\mathrm{B}}$

$$\langle 0 \rangle \quad (x' > 0 \wedge -8 \leq x' < 8 \wedge a' = 0 \wedge b' = 0) \rightarrow p^{\mathrm{I}}(x', a', b')$$

$$\langle 1 \rangle \quad \left(p^{\mathrm{I}}(x', a', b') \wedge a' < x'\right) \rightarrow p^{\mathrm{I}}\left(x', \mathrm{uts}_4\left((a' + 1) \bmod 16\right), \mathrm{uts}_4\left((b' - 1) \bmod 16\right)\right)$$

$$\langle 2 \rangle \quad \left(p^{\mathrm{I}}(x', a', b') \wedge \neg(a' < x')\right) \rightarrow q^{\mathrm{I}}(a', b')$$

Π_{B}:

$$\langle 3 \rangle \quad q^{\mathrm{B}}(a, b) \rightarrow \left(\left(a \oplus^{\mathrm{B}} b\right) <_{\mathrm{s}}^{\mathrm{B}} 0000\right)$$

Encoding, step 3: the integer fragment

i. Predicate renaming
ii. Variable renaming
iii. Constraint translations
iv. Domain restrictions

Π_{I}:

$x' \restriction_{\mathrm{B}}$

$$\langle 0 \rangle \quad (x' > 0 \wedge -8 \leq x' < 8 \wedge a' = 0 \wedge b' = 0) \rightarrow p^{\mathrm{I}}(x', a', b')$$

$$\langle 1 \rangle \quad \left(p^{\mathrm{I}}(x', a', b') \wedge a' < x'\right) \rightarrow p^{\mathrm{I}}\left(x', \sout{\mathrm{uts}_4((a'+1) \bmod 16)}, \sout{\mathrm{uts}_4((b'-1) \bmod 16)}\right)$$

$$\langle 2 \rangle \quad \left(p^{\mathrm{I}}(x', a', b') \wedge \neg(a' < x')\right) \rightarrow q^{\mathrm{I}}(a', b')$$

Π_{B}:

$$\langle 3 \rangle \quad q^{\mathrm{B}}(a, b) \rightarrow \left(\left(a \oplus^{\mathrm{B}} b\right) <_{\mathrm{s}}^{\mathrm{B}} 0000\right)$$

Optimization: **eliminate modulo and uts operations where overflow is impossible**

Encoding, step 4: interface constraints

Π_{I}:

$\langle 0 \rangle \quad (x' > 0 \wedge -8 \leq x' < 8 \wedge a' = 0 \wedge b' = 0) \rightarrow p^{\mathrm{I}}(x', a', b')$

$\langle 1 \rangle \quad \left(p^{\mathrm{I}}(x', a', b') \wedge a' < x'\right) \rightarrow p^{\mathrm{I}}(x', a' + 1, b' - 1)$

$\langle 2 \rangle \quad \left(p^{\mathrm{I}}(x', a', b') \wedge \neg(a' < x')\right) \rightarrow q^{\mathrm{I}}(a', b')$

Π_{B}:

$\langle 3 \rangle \quad q^{\mathrm{B}}(a, b) \rightarrow \left(\left(a \oplus^{\mathrm{B}} b\right) <_{\mathrm{s}}^{\mathrm{B}} 0000\right)$

$\Theta = \{q^{\mathrm{I}} \rightsquigarrow q^{\mathrm{B}}\}$

Semantics: derivations across fragments

Π_I:

$\langle 0 \rangle \quad (x' > 0 \wedge -8 \leq x' < 8 \wedge a' = 0 \wedge b' = 0) \rightarrow p^{\mathrm{I}}(x', a', b')$

$\langle 1 \rangle \quad \left(p^{\mathrm{I}}(x', a', b') \wedge a' < x'\right) \rightarrow p^{\mathrm{I}}(x', a' + 1, b' - 1)$

$\langle 2 \rangle \quad \left(p^{\mathrm{I}}(x', a', b') \wedge \neg(a' < x')\right) \rightarrow q^{\mathrm{I}}(a', b')$

Π_{B}:

$\langle 3 \rangle \quad q^{\mathrm{B}}(a, b) \rightarrow \left(\left(a \oplus^{\mathrm{B}} b\right) <_{\mathrm{s}}^{\mathrm{B}} 0000\right)$

$\Theta = \{q^{\mathrm{I}} \rightsquigarrow q^{\mathrm{B}}\}$

local

$p^{\mathrm{I}}(1,0,0)$

local

$p^{\mathrm{I}}(1,1,-1)$

local

$q^{\mathrm{I}}(1,-1)$

externa
l

$q^{\mathrm{B}}(0001,1111)$

Derivation

$0001 \oplus^{\mathrm{B}} 1111 = 1110 <_{\mathrm{s}}^{\mathrm{B}} 0000$

Query Check

End of lecture 2